

General Assignment of IPK Project

2024/2025

Welcome to the IPK projects guideline repository.

Table of Contents

- [IPK 2024/2025](#)
 - [Table of Contents](#)
 - [Goal](#)
 - [Coding Instruction](#)
 - [C/C++](#)
 - [C#](#)
 - [Submission Instructions](#)
 - [Documentation Instructions](#)
 - [Developer Environment and Virtualization Remarks](#)
 - [Evaluation](#)
 - [Project 1 - Multiple variants](#)
 - [Deadlines](#)
 - [Evaluation](#)
 - [Languages](#)
 - [Variants](#)
 - [Project 2 - Client for a chat server](#)
 - [Deadlines](#)
 - [Evaluation](#)
 - [Languages](#)
 - [Variants](#)

Goal

According to the assignment, you need to create an application communicating via a network.

You can implement what is not explicitly stated in the assignment according to your choice; however, describe the significant design decisions in the documentation.

Coding Instruction

You can develop Projects 1 and 2 in C/C++ and C#.

If you decide to implement the project in C, let it be by the C17 standard.

If you decide to implement the project in C++, let it be by the C++20 standard.

If you decide to implement the project in C#, make your implementation .NET 9+ compatible and out-of-the-box runnable on the reference virtual machine with the provided developer environment active.

The use of object-oriented programming is required when implementing the project in languages such as C++ or C#.

`Makefile` is mandatory for all project variants and programming languages.

Compilation of your projects will be done by calling `make`.

For C/C++, compiling with `Makefile` is native, for C#, you can call the standard `dotnet build` in the appropriate phony.

Using `Makefile` is to make the compilation of projects agnostic to the programming language used.

Among the allowed libraries are:

C/C++

- BSD sockets API
- `libpcap`
- `libnet`
- or any standard header files present on the reference machine (e.g., `arpa/inet.h`)
- any testing framework of your choice
- any utility library not directly solving the core of the project, assuming you include it directly with your source code

C#

- anything included in the base SDK
- any NuGet package not directly solving the core of the project (e.g., `xunit`, `Microsoft.Extensions.*`, `CommandLineParser`)
- `SharpPCAP`

The implementation should mark platform-dependent constructs (e.g., will only work on Linux).

Project source codes should be usable and meaningful, adequately annotated and formatted, and separated into appropriate methods and modules.

The program should include help to inform the user about the program's operation and parameters.

Any program errors should be intuitively described to the user.

The application shall not terminate with a `SEGMENTATION FAULT` or other violent system termination (e.g., division by zero).

An essential part of this project is demonstrating your software engineering skills.

Please use **private** [Git](#) repository on the faculty [Gitea portal](#) when versioning your project.

Make sure to give access to all [IPK instructors](#).

Your repository should have *multiple* commits (preferably using [semantic commit messages](#)) illustrating your development history.

Submission Instructions

The project archive (and repo as well) must contain at least the following mandatory files:

- all project source codes and stand-alone libraries;
- working `Makefile`, which will compile or build the executable binary (with the filename specified in the assignment) from the source codes;
- documentation written in Markdown in the file `README.md`;
- the license in the file `LICENSE` with the most restrictive (i.e., preserving maximum rights of you as the author) commonly recognized open-source license you can find;
- `CHANGELOG.md` will briefly describe implemented functionality and known limitations (a documented problem is better than an undocumented one).

If you are unaware of any limitations, state that in the changelog as well.

All of the aforementioned files are expected to be found **in the root directory** of the project, except for the source codes and used libraries – these should be located in appropriate directories as per the programming language used.

Do not submit any archives and other binary or build-generated files.

Please verify the following steps for the successful submission of your projects:

1. check whether your code successfully compiles/builds under the reference developer environment and check the filename syntax of mandatory files;
2. upload ZIP (and only ZIP) archive with a filename matching `xlogin99.zip`, containing the contents of your Gitea repo with all mandatory files (see above);
3. give access to your private Gitea repo to all members of [NES@FIT Gitea group](#).

We will ignore any submission via email (or any other communication channel) after the deadline.

Documentation Instructions

Good documentation should include the following:

- content structuring;

- executive summary of the theory necessary to understand the functionality of the implemented application;
- UML diagrams or at least a narrative about interesting source code sections;
- most importantly, the testing part, which should prove that you have done some validation and verification of implemented functionality;
- description of extra functionality or features beyond the standard outlined by the assignment;
- bibliography with the list of all the sources used.

Documentation can be in the following languages:

- Czech,
- Slovak,
- or English.

Source code comments must be in the following language: English.

The section of the documentation describing testing should be sound and reproducible (e.g., an accurate description of network topology, involved hardware specification and software versions). Testing should include at least a few runs of your application testing proper functionality and various edge cases.

Please, compare your project with another similar tool (if such tool exists).

Since it is your application, you should also be the author of test sets (i.e., avoid using someone else's tests).

Testing proofs can have textual program outputs (screenshots for textual information are undesired). Presenting testing inputs/outputs is not enough; a good testing section should describe the following:

- what was tested
- why it was tested
- how it was tested
- what was the testing environment
- what were the inputs, expected outputs, and actual outputs

When working with any bibliography or sources online, please cite according to [faculty guidelines](#).

When adopting or inserting snippets of someone else's source code into your project, you need to use them according to the author's license terms.

Moreover, you need to mark this source code and reference the author explicitly in your source files.

Avoid any unidirectional or bidirectional plagiarism!

Developer Environment and Virtualization Remarks

The individual assignment specifies a custom reference environment located in [its own repository](#).

The reference environment is separate from the provided virtual machine and can be activated on-

demand for a particular language, mainly to avoid wasting resources with unnecessary language toolchains.

To utilize the developer environment, you can use the virtual machine(s) downloadable from [FIT Nextcloud](#)

(hosted by, e.g.,

[VirtualBox](#)

or [Hyper-V](#)

or [VMWare Player](#)

or [Parallels](#) depending on your platform).

The user login is `ipk` with password `ipk`.

Projects will be evaluated on the reference virtual machine [IPK25_Ubuntu24.ova](#) and corresponding architecture (Ubuntu 24.04 LTS, `amd64`).

In case you do not have access to the corresponding architecture/platform you can always leverage services of [CVT at FIT](#).

These are the developer environments acceptable for this year:

- `c`
- `csharp`

To activate or modify the environment, follow the guide in the corresponding [developer environment repository](#).

However, your *Makefile* should work with the default, unedited environment.

Due to the nature of the projects, you can assume that:

- the project will be run and evaluated with root privileges,
- the reference machine will be connected to the Internet.

Evaluation

The standard criteria for the evaluation can be seen below.

Point percentages are relative to the grading section of the task (e.g. `50%` of a documentation task for `6pt` is `3pt`).

- code and application behaviour:
 - unstructured, uncommented, single-file or spaghetti source code = up to `-20%`
 - non-functioning or missing Makefile = 0 points (`-20%` if fixed)
 - wrong filenames or location of mandatory files = up to `-10%` points
 - temporary or irrelevant files inside the submitted archive = up to `-10%` points

- incorrect dynamic memory handling (e.g., memory leaks, unclosed file descriptors) = up to **-25%**
- input/output of the program is not in line with the assignment requirements = up to **-50%**
- documentation:
 - missing or poor documentation = up to **-100%** points
 - wrong bibliography or poor formatting = up to **-33%** points
 - insufficient proof of testing of the project = up to **-66%** points
- general criteria:
 - the project cannot be compiled, executed or tested = 0 points
 - the program ends abruptly (e.g., **SEGFault** , **SIGSEGV**) = 0 points
 - the project solution is not in compliance with the assignment = 0 points
 - plagiarism = 0 points for all involved parties and personal meeting with teacher, guarantor and expel committee

The archive submitted via IS VUT will be the basis for your evaluation.

Files in the Gitea repo are additional content accompanying the submission but take no precedence over the content in IS VUT.

Therefore, any technical difficulties on the Gitea side will not affect your score.

Project 1 – Multiple variants

Deadlines

Date	Description
22. 3. 2025 23:59	Registrations end
27. 3. 2025 23:59	Submission deadline

Evaluation

You can get up to 7 points for working applications in compliance with the assignment, as well as demonstrating good programming skills.

You can get up to 3 points for documentation.

You can earn extra points for significant extension of your program's features.

You cannot get more than 10 points from the project, but the points for extra effort can help you minimize any losses during the assesment of standard requirements.

Languages

- C
- C++
 - Object-oriented programming **is mandatory**
- C#
 - Object-oriented programming **is mandatory**

Variants

- ALPHA: Last year's points from Project 2
 - only points from the last academic year 23/24 are accepted;
 - last year's score x , $(x / 20) * 10$ is accepted.
- [DELTA: L2/L3 scanner](#)
- [OMEGA: L4 scanner](#)

Project 2 – Client for a chat server

Deadlines

Date	Description
19. 4. 2025 23:59	Registrations end
20. 4. 2025 23:59	Submission deadline

Evaluation

You can get up to 24 points for working applications in compliance with the assignment, as well as demonstrating good programming skills.

You can get up to 6 points for documentation.

You can earn extra points for significant extension of your program's features.

You cannot get more than 30 points from the project, but the points for extra effort can help you minimize any losses during the assesment of standard requirements.

Languages

- C
- C++
 - Object-oriented programming **is mandatory**
- C#
 - Object-oriented programming **is mandatory**

Variants

- Last year's points from Project 1
 - only points from the last academic year 23/24 are accepted;
 - last year's score $x < 15.0p$, x is accepted;
 - last year's score $x \geq 15.0p$, $(x / 20) * 30$ is accepted.
- [Client for a chat server using the IPK25-CHAT protocol](#)